id: MDCS-TICKET-003
title: Implement data anonymizer utility.
epic: epic-dumb-copilot
status: ready
risk: medium
allowed_areas:
- tools/data_anonymizer.py
- tools/requirements.txt
must_not_touch:
- procedures/
- references/
requirements:
- Create `tools/data_anonymizer.py`.
- Accept command-line arguments for input file (CSV/XLSX), output file, columns to mask, and masking method.
- Implement the following masking strategies:
1. **Random String/Numeric Mask**: Replace values with fixed formats (e.g., `ANON_STR_1`, `ANON_NUM_10`).
2. **Hash-Mapping (Join-Preserving)**: Ensure matching values in a column (like `CUSTOMER_ID`) are consistently mapped to th
3. **Numerical Perturbation**: Add random noise (e.g., ±5-15%) to numeric columns (like revenue or transaction amount) to m
4. **Common PII Detectors**: Provide simple regular-expression fallback detection for columns containing emails, telephone nur
- Output a clean, fully anonymized CSV/file ready to be pasted.
non_goals:
- Do not use heavyweight external NLP libraries (e.g., Spacy/Presidio) to ensure the script stays lightweight and easy to run in a
acceptance_criteria:
- Running the anonymizer on a test CSV successfully masks defined columns.
- Joins between two anonymized CSV outputs (e.g., customers and orders) are preserved.
verification_commands:
- python tools/data_anonymizer.py –help
depends_on:
- MDCS-TICKET-001

Body

Goal

Create a python utility script that replaces sensitive data with realistic, join-preserving dummy values locally on the user's workstation.

Context

Copilot must never see real data. Pasting raw data rows is a violation of data privacy policies. A local data anonymizer masks all sensitive text and numeric variables while maintaining data relationships, allowing you to feed safe dummy samples to Copilot for debugging.

Procedure

1. Add `pandas` and `openpyxl` to `tools/requirements.txt`.
2. Write `tools/data_anonymizer.py`.
3. Implement a deterministic salt-hashed dictionary mapping for keys (e.g. mapping `Tristan` to `User_A` and keeping it consistent across rows).
4. Implement numerical perturbation using python's `random` package.
5. Create a test script with a small mock data table containing names, emails, values, and IDs, run the tool, and assert that no raw values remain in the output.